

A

Project Report On

“CodeInsight-Code Reviewer”

Submitted By

Meet Lathidadiya (MCA25107)

Under the guidance of

Prof. Deepak Pandita

Assistant Professor

MASTERS OF COMPUTER APPLICATION

SCHOOL OF COMPUTER APPLICATIONS

PIMPRI CHINCHWAD UNIVERSITY

ACADEMIC YEAR 2025-26

ACKNOWLEDGEMENT

I would like to express my gratitude and sincere regards to the following people to whom I am grateful for their support and help without which I would not have been able to do this project entitled “**CodeInsight**”.

I would like to thanks **Dr. Kamlesh Meshram**, HOD, SOCA, PCU for providing us with excellent facilities valuable guidance.

My special thanks to **Prof. Deepak Pandita**, Internal Guide, whose cooperation and keen interest coupled with valuable suggestions help me in the completion of the project.

Last but not least, I am thankful to all Teaching and Non-Teaching staff of the college, whose names have not been mentioned here but have directly or indirectly helped me in the completion of this project.

Meet Lathidadiya

Abstract

Project Title: CodeInsight-Code Reviewer

CodeInsight is a web-based code quality analysis system developed using Flask that helps users upload and analyze source code files (mainly Python and Java). It evaluates code using key quality factors such as syntax correctness, style compliance, complexity, and maintainability, then provides a score, issue list, improvement suggestions, and downloadable PDF report.

Problem Statements are, In student projects and early-stage software development, code quality is often judged manually, which is time-consuming and inconsistent. Many developers focus on getting output rather than writing clean, maintainable, and standard-compliant code. Existing tools are either too technical for beginners or scattered across multiple platforms, making regular quality checks difficult.

Objectives are, To develop a web-based code analysis platform that enables users to upload Python and Java source files and receive automatic quality evaluation in a simple and accessible way, To improve the overall quality of code by identifying syntax errors, style violations, complexity issues, and maintainability problems before the code is submitted or used in a project, To provide clear, explainable scores, issue breakdowns, and improvement suggestions so that students and developers can understand the strengths and weaknesses of their code.

This project was built in small iterative cycles. Each cycle delivered a working feature set, then refined it with feedback, testing, and bug fixes. The methodology combines SDLC phases with a practical pipeline for code analysis.

Key outcomes of, CodeInsight are the automation of code quality evaluation for Python and Java files, the detection of syntax, style, complexity, and maintainability issues, and the generation of explainable scores with detailed feedback. It also provides secure user authentication, plan-based access to different languages, file upload and analysis tracking, actionable improvement suggestions, and professional PDF report generation. Overall, the system delivers a user-friendly web platform that helps users review, understand, and improve source code quality

Declaration

I hereby declare that the project work entitled, **CodeInsight**-Code Reviewer submitted under guidance of **Prof. Deepak Pandita**, is my original work completed under the four walls of **Pimpri Chichwad University**.

The Project Report submitted is my own work and has not been duplicated from any other source. I shall be responsible for any unpleasure moment/situation.

Meet Lathidadiya

INDEX

Sr No	Description	Page No
1.	Introduction Background/Problem Statement Objectives of the Project Scope of the Project Organization of the Report	1
2.	Literature Survey/Review Overview of existing work Projects, or products reviewed Identification of gaps	7
3.	System Analysis Requirements Specification Feasibility Study Tools and Technology Used	10
4.	Design System Architecture Diagrams Database Design	14
5.	Implementation Description of Modules/Components Screenshots of the Application	20
6.	Testing Test Cases Bug Fixes/Issues Resolved	28
7.	Results and Discussion Outcomes of the project Comparison with existing solutions Challenges Faced	33
8.	Conclusion and Future Scope Summary of the work Limitations Future Enhancements	37
	Refernces	39

1. Introduction

➤ Background/Problem Statement:-

In student projects and early-stage software development, code quality is often judged manually, which is time-consuming and inconsistent. Many developers focus on getting output rather than writing clean, maintainable, and standard-compliant code. Existing tools are either too technical for beginners or scattered across multiple platforms, making regular quality checks difficult.

- **Clearly Define the Problem:**

Developers and students often submit code that runs but does not meet quality standards like proper syntax, coding style, low complexity, and maintainability

Manual review of these aspects is slow, inconsistent, and difficult to scale, especially when many files or users are involved

As a result, code quality issues are detected late, reducing software reliability and increasing rework effort

- **Issues in Existing System:**

Existing code review approaches are fragmented and not beginner-friendly

Different tools are needed for syntax checking, style analysis, complexity metrics, and reporting, which makes the process time-consuming and hard to manage

Many systems also lack a unified dashboard, clear scoring model, actionable suggestions, and downloadable reports, making continuous improvement difficult for learners and teams

➤ Objectives of the Project:

1. Automated Code Quality Analysis:

The primary objective of CodeInsight is to automatically analyze source code and evaluate its quality using multiple parameters such as syntax correctness, coding style, complexity, and maintainability.

- Eliminates manual code review effort
- Provides instant feedback to developers
- Helps beginners understand code quality standards

2. Intelligent Scoring System:

To design a **mathematical scoring model** that evaluates code based on:

- Style (45%)
- Complexity (25%)
- Maintainability (25%)
- Structure (5%)

The system uses:

- Continuous scoring formulas
- Issue density normalization
- Weighted penalties

3. Multi-Level Code Inspection:

The project performs analysis at multiple levels:

- Syntax Analysis → Detect errors using AST
- Style Analysis → Using pylint rules
- Complexity Analysis → Using Radon
- Semantic Analysis → Custom logic (inefficiencies, bad practices)

4. Maintainability & Technical Debt Estimation:

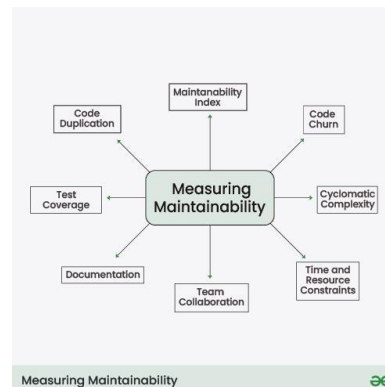
To calculate:

- Maintainability Index (MI)

- Technical Debt (in hours)

This helps developers:

- Understand long-term code quality
- Identify risky or hard-to-maintain code



5. User-Friendly Dashboard Interface:

To provide a modern UI that displays:

- Overall score
- Category-wise metrics
- Suggestions
- Visual indicators

6. Support for Multiple Languages (Extensible):

The system supports:

- Python (fully implemented)
- Java (partially implemented)
- Extensible for JS, HTML, CSS

➤ Scope of Project:

• Target Users:

The target users of CodeInsight are students, beginner programmers, software developers, and faculty members who need a simple way to analyze and improve code quality. It is especially useful for users working on Python and Java projects.

- **Application Area:**

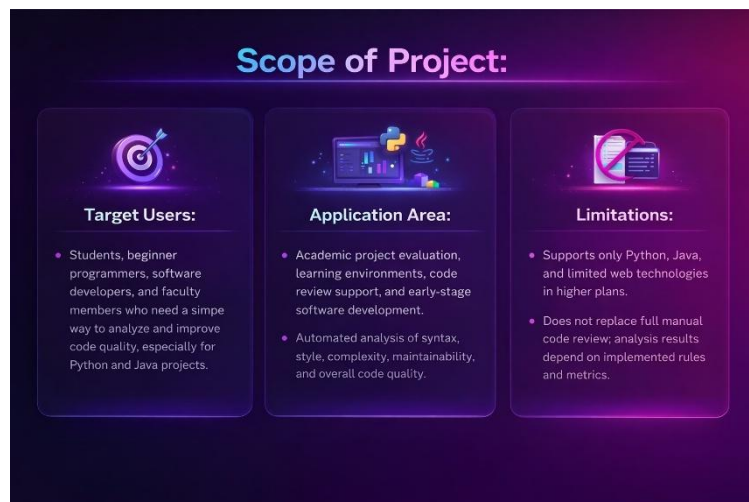
This project can be applied in academic project evaluation, learning environments, code review support, and early-stage software development.

It helps in checking syntax, style, complexity, maintainability, and overall code quality through automated analysis and reporting.

- **Limitations:**

The system currently supports only selected programming languages such as Python and Java, with limited support for some web technologies in higher plans.

It does not replace full manual code review, and the analysis results depend on the rules and metrics implemented in the system.



➤ **Organization of the Report:**

The report is organized into seven chapters covering introduction, literature review, system design, implementation, testing, results, and conclusion to provide a comprehensive understanding of the CodeInsight system.

- **Chapter 1: Introduction**

This chapter presents an overview of the **CodeInsight – Intelligent Code Analysis & Scoring System**. It includes the background of code quality analysis, problem statement, objectives, and scope of the system. It explains

the need for an automated platform that helps developers analyze and improve code quality efficiently.

- **Chapter 2: Literature Review / Existing System**

This chapter discusses existing tools such as static code analyzers (e.g., pylint, Radon) and traditional manual code review methods. It highlights their features and limitations, such as lack of unified scoring, limited semantic analysis, and absence of user-friendly dashboards. It also emphasizes the need for an integrated and intelligent analysis system like CodeInsight.

- **Chapter 3: System Analysis and Design**

This chapter explains the system architecture and design of CodeInsight. It includes:

- System architecture diagram
- Data Flow Diagram (DFD)
- Module design (Code Analyzer, Scoring Engine, Suggestion Engine, UI)
- Database/File structure
- Workflow of the analysis pipeline (syntax → style → complexity → scoring)

- **Chapter 4: System Implementation**

This chapter describes the technologies used, such as Python, Flask, Pylint, Radon, HTML, CSS (Tailwind), and JavaScript. It explains how:

- Code analysis is performed using AST and external tools
- Scoring algorithm is implemented
- Suggestions are generated
- Dashboard UI is developed and integrated with backend

- **Chapter 5: Testing and Validation**

This chapter explains the testing methods used to ensure correctness and reliability of the system. It includes:

- Unit testing of analysis modules
- Validation of scoring algorithm
- Testing with different types of code files
- Performance and accuracy testing

- **Chapter 6: Results and Discussion**

This chapter presents the final output of the system along with screenshots of the dashboard and analysis reports. It discusses:

- Score interpretation
- System performance
- Accuracy of analysis
- User experience and usability

- **Chapter 7: Conclusion and Future Scope**

This chapter summarizes the project, key achievements, and learning outcomes. It also discusses future improvements such as:

- Support for more programming languages
- AI-based code suggestions
- Integration with IDEs (VS Code plugin)
- Real-time analysis and collaboration features

2. Literature Survey/Review

➤ Overview of existing work:

Existing work in code quality analysis includes tools like Pylint and Radon, manual code reviews, and platforms such as SonarQube. These tools help detect errors, enforce coding standards, and measure complexity and maintainability. Static analysis tools provide quick automated results, while manual reviews help identify deeper logical issues. Platforms like SonarQube also offer dashboards and continuous monitoring for large projects.

However, these systems have limitations. They often focus on specific aspects of code rather than providing a complete evaluation and lack a unified scoring system. Many tools are complex and not suitable for beginners. CodeInsight addresses these issues by combining multiple analysis techniques into a simple, user-friendly system that provides an overall score and helpful suggestions to improve code quality.

➤ Projects or Products Reviewed:

- **Pylint**

Pylint is a popular static code analysis tool for Python that checks for coding standards, detects errors, and identifies bad practices. It provides detailed reports based on predefined rules. However, it mainly focuses on style and syntax without offering a unified scoring system.

- **Radon**

Radon is used to measure code complexity and maintainability index in Python programs. It helps developers understand how complex or maintainable their code is. However, it only focuses on metrics and does not provide suggestions or overall code evaluation.

- **SonarQube**

SonarQube is a widely used platform for continuous inspection of code quality. It detects bugs, vulnerabilities, and code smells while providing visual dashboards and reports. Despite its powerful features, it is complex to set up and may not be suitable for beginners or small projects.

- **Manual Code Review**

Manual code review is a traditional approach where developers review code to identify issues and improve quality. While it provides deep insights, it is time-consuming, subjective, and not scalable for large codebases.

➤ **Identification of gaps:**

The analysis of existing systems reveals the need for an integrated, user-friendly, and intelligent code analysis platform. CodeInsight addresses these gaps by providing a unified scoring system, semantic analysis, clear suggestions, and a modern interface suitable for both students and developers.

- **Lack of Unified Scoring System**

Most existing tools provide separate outputs such as errors, warnings, or complexity values, but they do not combine these metrics into a single, easy-to-understand score. This makes it difficult for users, especially beginners, to evaluate overall code quality.

- **Limited Semantic Analysis**

Many tools focus mainly on syntax and style checking but fail to detect deeper issues such as inefficient algorithms, poor logic design, or bad coding practices. This limits their ability to provide meaningful insights into real code quality.

- **Complexity of Tools**

Advanced platforms like SonarQube and DeepSource require complex setup, configuration, and integration with CI/CD pipelines. This makes them

less suitable for students, small projects, or beginners who need a simple solution.

- **Lack of User-Friendly Interface**

Most tools provide output in terminal or complex dashboards, which may not be easy to understand. There is a lack of simple and intuitive interfaces that clearly present results and suggestions.

- **No Integrated Suggestion System**

Existing tools often list issues but do not provide clear, actionable suggestions or guidance on how to improve the code, especially in a beginner-friendly manner.

- **Fragmented Analysis Approach**

Different tools are required for different tasks (e.g., Pylint for style, Radon for complexity), leading to a fragmented workflow. There is no single platform that integrates all aspects of code analysis.

3. System Analysis

➤ Requirements Specification:

The requirements specification defines the functional and non-functional requirements of the CodeInsight – Intelligent Code Analysis & Scoring System. These requirements describe what the system should do and how it should perform.

❖ Functional Requirements

Functional requirements describe the core features and operations of the system.

- The system shall allow users to upload or input source code files for analysis.
- The system shall detect the programming language based on file extension.
- The system shall perform syntax analysis to identify errors in the code.
- The system shall perform style checking using tools like Pylint.
- The system shall calculate code complexity using appropriate metrics.
- The system shall evaluate maintainability index (MI) of the code.
- The system shall perform semantic analysis to detect inefficient logic and poor coding practices.
- The system shall generate a unified score out of 100 based on multiple parameters.
- The system shall display detailed results, including:
 - Style score
 - Complexity score
 - Maintainability score
 - Structure score
- The system shall provide suggestions and recommendations to improve code quality.

- The system shall handle empty or invalid files and return appropriate messages.
- The system shall allow users to view analysis results through a dashboard interface.

❖ **Non-Functional Requirements**

Non-functional requirements describe the performance, usability, and quality attributes of the system.

- The system should provide fast analysis results with minimal delay.
- The system should be user-friendly and easy to understand, especially for beginners.
- The system should ensure accuracy and reliability in code analysis and scoring.
- The system should be scalable to support additional programming languages in the future.
- The system should have a modular architecture for easy maintenance and updates.
- The system should be secure and handle user data safely.
- The system should be portable, allowing deployment on different platforms.
- The system should maintain consistency in scoring and results.
- The system should be extensible to integrate with other tools or IDEs.
- The system should have a responsive and modern user interface.

➤ **Feasibility Study:**

The feasibility study evaluates the practicality of developing and implementing the CodeInsight – Intelligent Code Analysis & Scoring System. It analyzes the project from technical, economic, and operational perspectives.

❖ **Technical Feasibility**

The proposed system is technically feasible as it is built using widely available and well-supported technologies such as Python, Flask, HTML, CSS (Tailwind), and JavaScript. Existing libraries like Pylint and Radon are used for code analysis, reducing development complexity.

- The required development tools are easily accessible
- The system does not require high-end hardware
- Integration of analysis tools is straightforward
- The architecture is modular and scalable

Te proposed system is technically feasible as it is built using widely available and well-supported technologies such as Python, Flask, HTML, CSS (Tailwind), and JavaScript. Existing libraries like Pylint and Radon are used for code analysis, reducing development complexity.

- The required development tools are easily accessible
- The system does not require high-end hardware
- Integration of analysis tools is straightforward
- The architecture is modular and scalable

❖ **Economic Feasibility**

The project is economically feasible as it requires minimal cost for development and deployment.

- Most tools and technologies used are open-source and free
- No expensive hardware or licenses are required
- Development can be done on a standard computer
- Maintenance cost is low

❖ **Operational Feasibility**

The system is operationally feasible as it is designed to be simple and user-friendly.

- Easy to use for students, developers, and faculty
- Requires minimal training
- Provides clear outputs and suggestions

➤ **Tools and Technology Used:**

❖ **Frontend:**

- HTML
- CSS
- JavaScript
- Bootstrap
- Chart.js

❖ **Backend:**

- Python
- Flask
- Flask-Mail
- Werkzeug
- Jinja

❖ **Database:**

- MySQL
- Flask-MySQLdb/MySQLdb connector

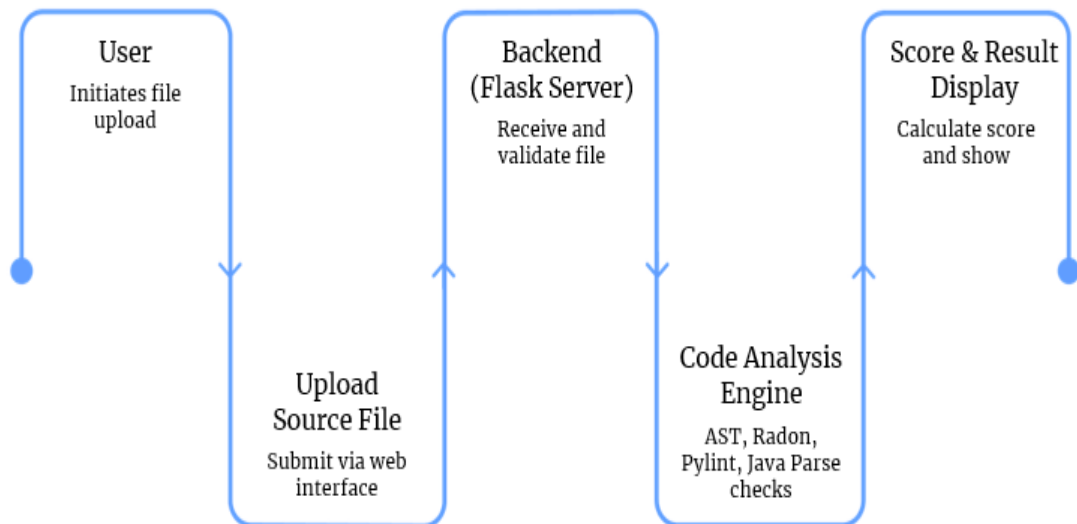
❖ **Tools/Platforms:**

- Pylint (style analysis)
- Java Parse (style analysis)
- Radon (complexity & maintainability)
- AST module (syntax analysis)
- ReportLab (PDF report generation)
- Git/GitHub
- VS Code

4. Design

➤ System Architecture:

The CodeInsight system architecture follows a simple pipeline-based design consisting of three main components: User Interface, Backend Server, and Code Analysis Engine.



- **UserLayer**

The process starts when the user uploads a source code file through the web interface. This provides an easy and interactive way to submit code for analysis.

- **Backend(FlaskServer)**

The backend receives the uploaded file, validates it, and determines the programming language. It acts as the central controller that manages the flow of data between different modules.

- **Code Analysis Engine**

The backend sends the file to the analysis engine, where multiple tools and techniques are applied:

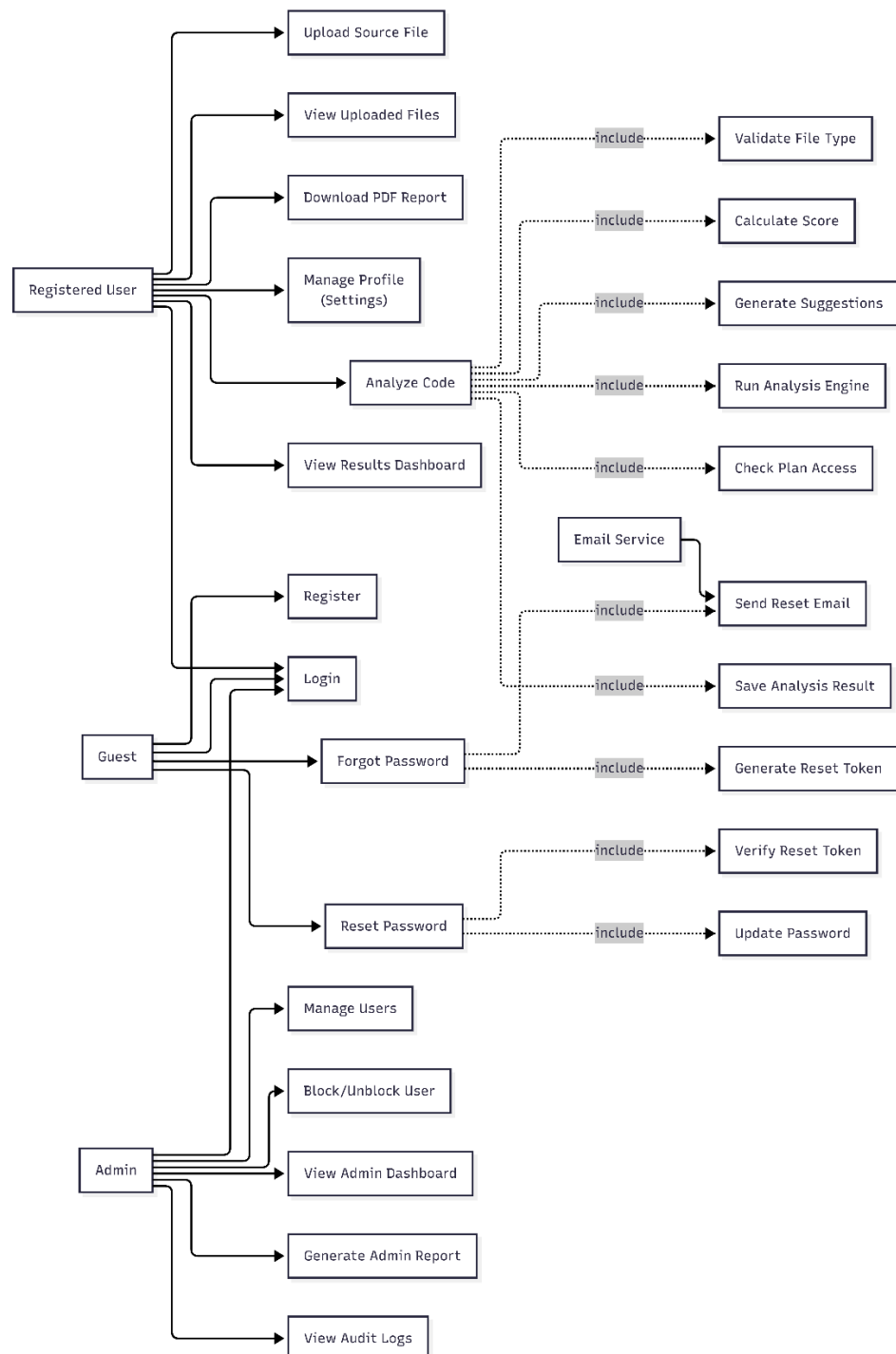
- AST for syntax checking
- Pylint for style analysis
- Radon for complexity and maintainability
- Java parser for Java files

- **Score & Result Display**

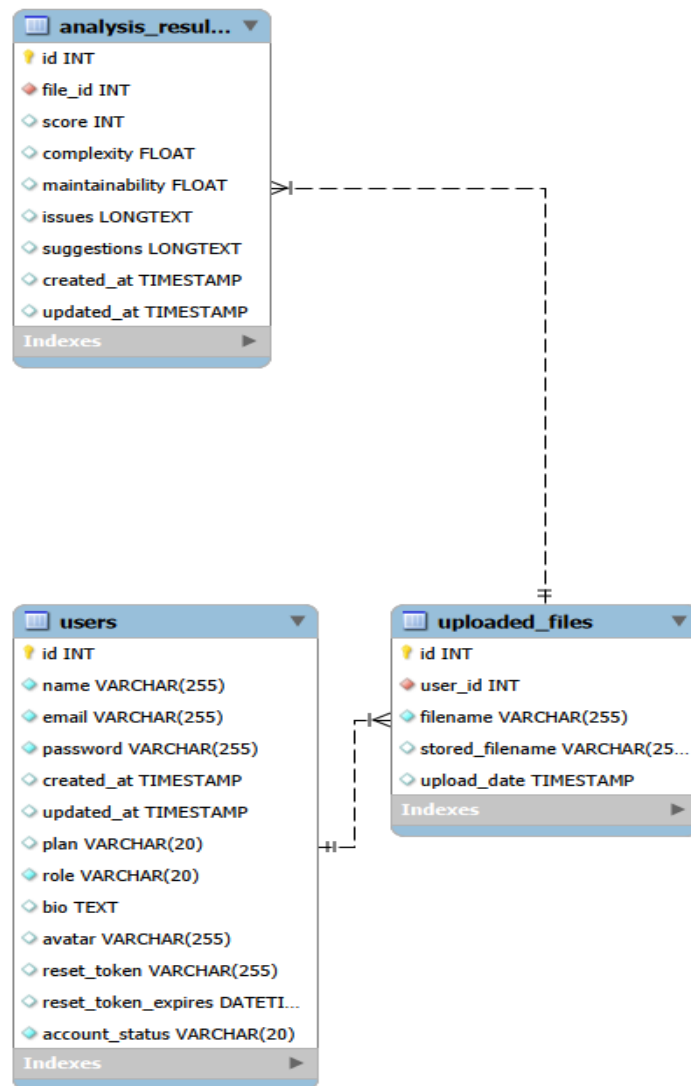
After analysis, the system calculates scores based on different metrics and displays the results, including suggestions, on the dashboard.

➤ **Diagrams:**

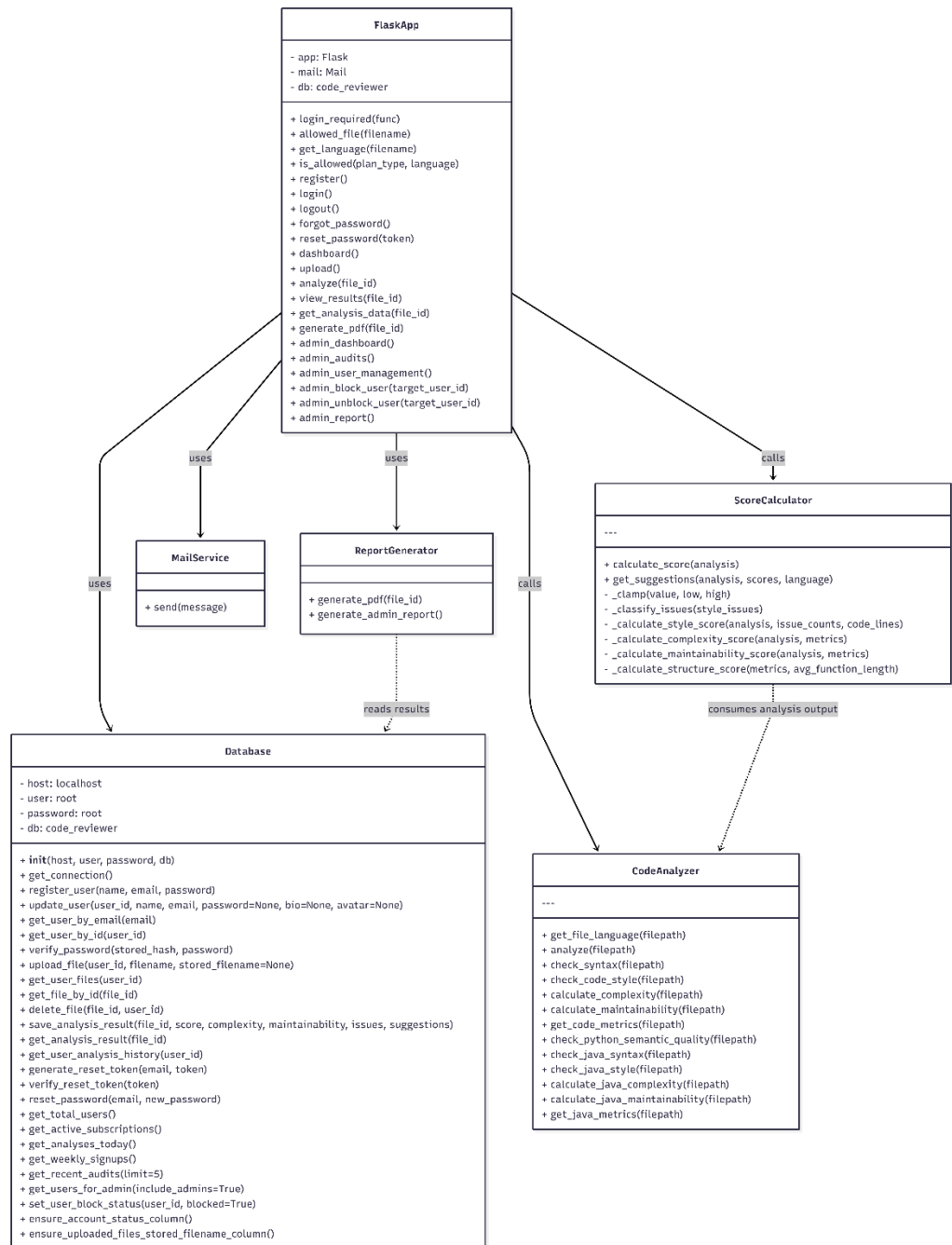
- **Use Case:**



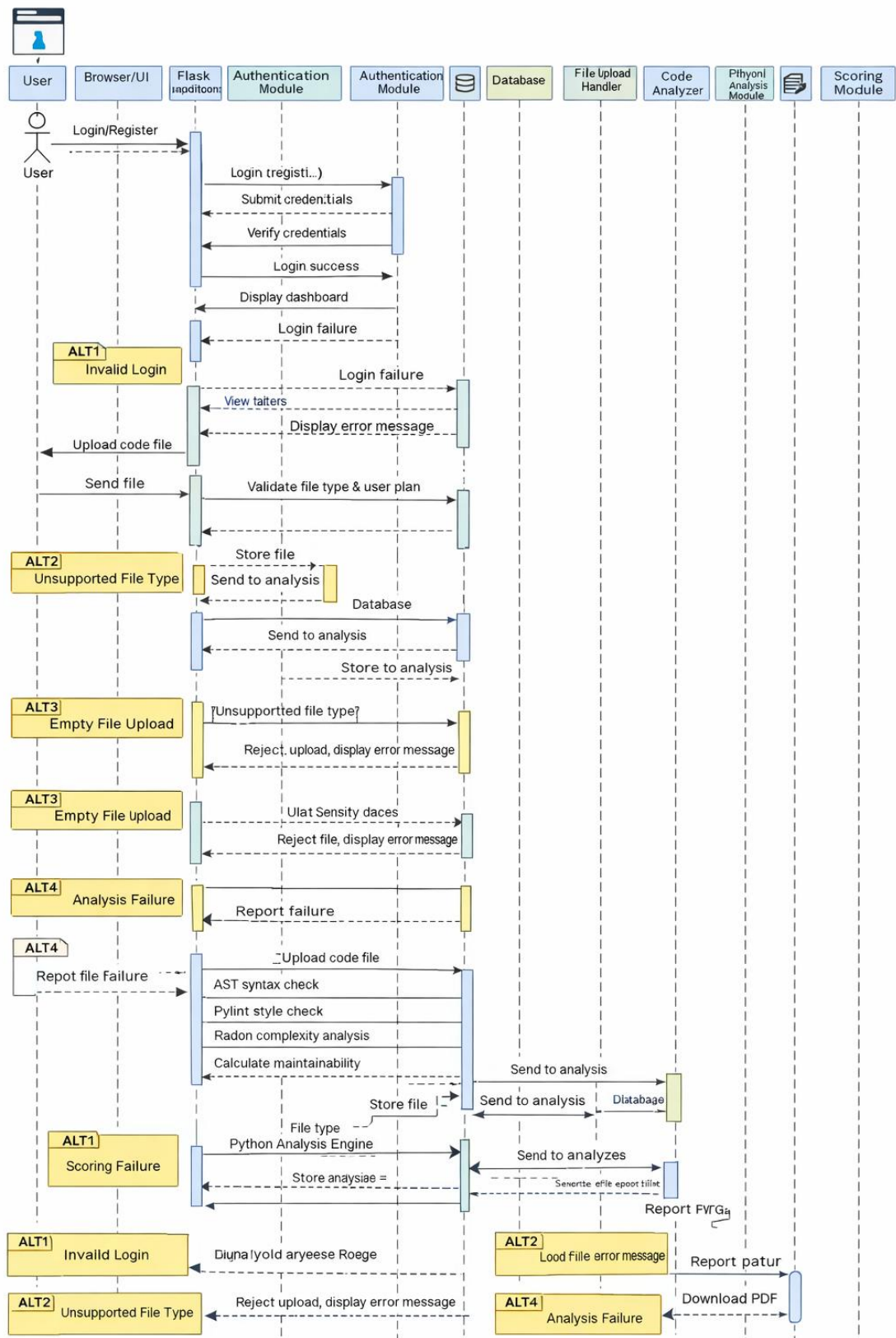
- **E-R Diagram:**



- **Class Diagram:**



- Sequence Diagram:



➤ Database Design:

• Users Table:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
🔑 id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
🔗 name	VARCHAR(255)	☐	☒	☐	☐	☐	☐	☐	☐	
🔗 email	VARCHAR(255)	☐	☒	☒	☐	☐	☐	☐	☐	
🔗 password	VARCHAR(255)	☐	☒	☐	☐	☐	☐	☐	☐	
🔗 created_at	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	CURRENT_TIMESTAMP
🔗 updated_at	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	CURRENT_TIMESTAMP ON...
🔗 plan	VARCHAR(20)	☐	☐	☐	☐	☐	☐	☐	☐	'free'
🔗 role	VARCHAR(20)	☐	☒	☐	☐	☐	☐	☐	☐	'user'
🔗 bio	TEXT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
🔗 avatar	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	'default.png'
🔗 reset_token	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
🔗 reset_token_expires	DATETIME	☐	☐	☐	☐	☐	☐	☐	☐	NULL
🔗 account_status	VARCHAR(20)	☐	☒	☐	☐	☐	☐	☐	☐	'active'
		☐	☐	☐	☐	☐	☐	☐	☐	

• Upload_files Table:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
🔑 id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
🔗 user_id	INT	☐	☒	☐	☐	☐	☐	☐	☐	
🔗 filename	VARCHAR(255)	☐	☒	☐	☐	☐	☐	☐	☐	
🔗 stored_filename	VARCHAR(255)	☐	☐	☐	☐	☐	☐	☐	☐	NULL
🔗 upload_date	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	CURRENT_TIMESTAMP
		☐	☐	☐	☐	☐	☐	☐	☐	

• Analysis_result Table:

Column Name	Datatype	PK	NN	UQ	B	UN	ZF	AI	G	Default/Expression
🔑 id	INT	☒	☒	☐	☐	☐	☐	☒	☐	
🔗 file_id	INT	☐	☒	☒	☐	☐	☐	☐	☐	
🔗 score	INT	☐	☐	☐	☐	☐	☐	☐	☐	'0'
🔗 complexity	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	'0'
🔗 maintainability	FLOAT	☐	☐	☐	☐	☐	☐	☐	☐	'0'
🔗 issues	LONGTEXT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
🔗 suggestions	LONGTEXT	☐	☐	☐	☐	☐	☐	☐	☐	NULL
🔗 created_at	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	CURRENT_TIMESTAMP
🔗 updated_at	TIMESTAMP	☐	☐	☐	☐	☐	☐	☐	☐	CURRENT_TIMESTAMP ON...
		☐	☐	☐	☐	☐	☐	☐	☐	

5. Implementation

➤ **Description of Modules/Components:**

The system includes modules such as authentication, admin control, frontend UI, backend server, file handler, code analyzer, language-specific engines, scoring module, suggestion module, database, and report generator, each contributing to efficient code analysis and management.

1. Authentication Module

This module handles user registration and login functionality.

- Verifies user credentials
- Manages user sessions
- Ensures secure access to the system

2. User Interface (Frontend)

This module provides interaction between the user and the system.

- Allows users to upload code files
- Displays analysis results and scores
- Provides a dashboard with visual insights

3. Flask Backend Server

This acts as the central controller of the system.

- Receives user requests
- Handles file uploads
- Communicates with analysis modules
- Sends results back to UI

4. File Upload Handler

This module processes uploaded files.

- Validates file type (Python/Java)
- Checks for empty or invalid files
- Stores files temporarily for analysis

5. Code Analyzer Module

This is the core module responsible for coordinating code analysis.

- Identifies programming language
- Routes code to appropriate analysis engine
- Aggregates analysis results

6. Python Analysis Engine

This module analyzes Python code using multiple tools.

- Syntax checking using AST
- Style analysis using Pylint
- Complexity calculation using Radon
- Maintainability evaluation

7. Java Analysis Engine

This module analyzes Java code.

- Syntax validation (basic parsing)
- Style checking using custom rules
- Complexity calculation
- Maintainability scoring

8. Scoring Module

This module calculates the overall code quality score.

- Combines style, complexity, maintainability, and structure scores
- Uses weighted formulas
- Generates a final score out of 100

9. Suggestion Module

This module generates improvement recommendations.

- Identifies issues in code
- Provides actionable suggestions
- Helps users improve code quality

10. Database Module

This module manages data storage.

- Stores user information
- Saves analysis results and reports
- Maintains history of uploads

11. Report Generator Module

This module generates detailed reports.

- Converts analysis results into readable format
- Generates downloadable reports (PDF)
- Includes score breakdown and suggestions

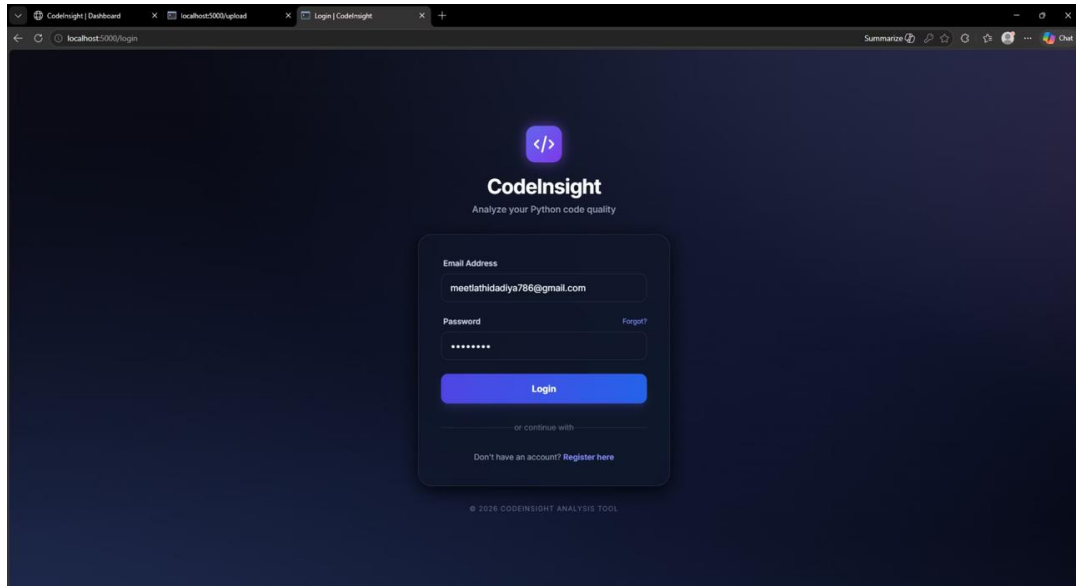
12. Admin Module

This module provides administrative control over the system.

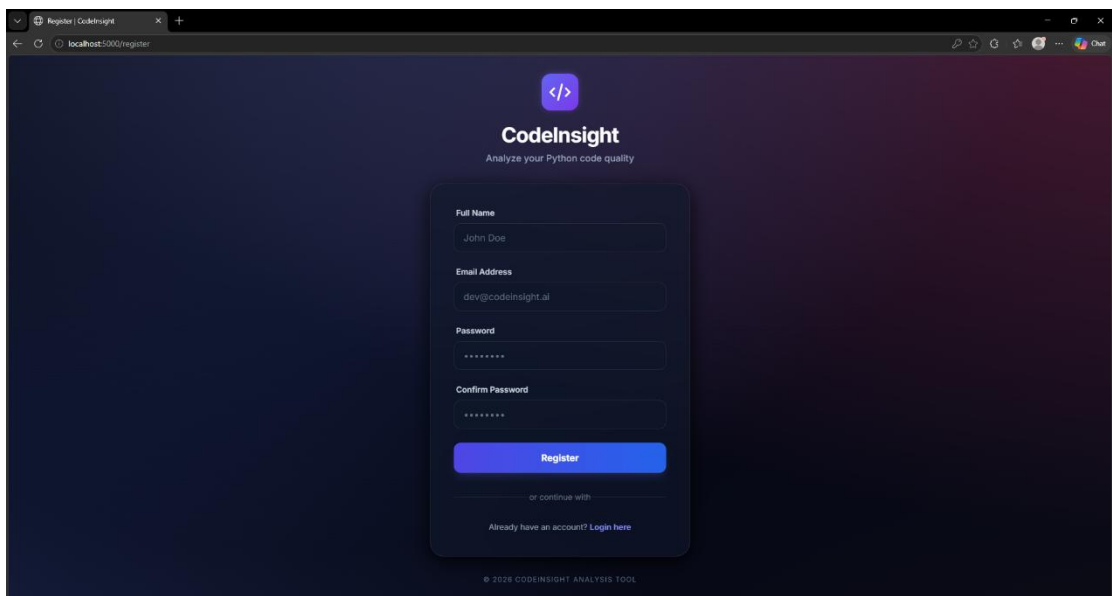
- Manage users (view, delete, control access)
- Monitor uploaded files and analysis history
- Configure system rules or scoring parameters (optional)
- View system statistics and usage

➤ Screenshots of the Application/System

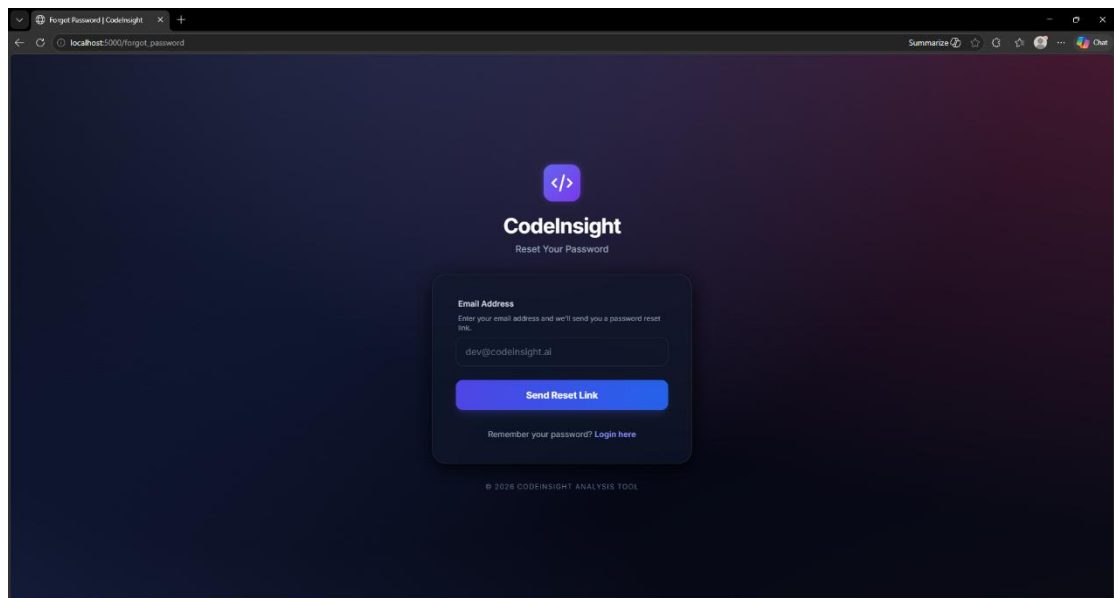
- **Login Page**



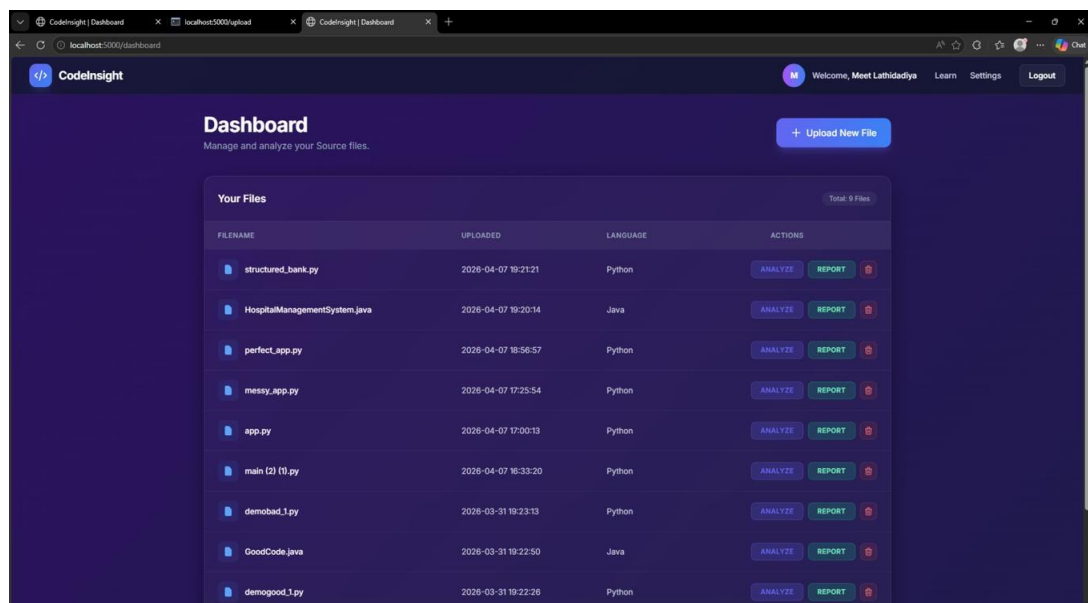
- **Signup Page**



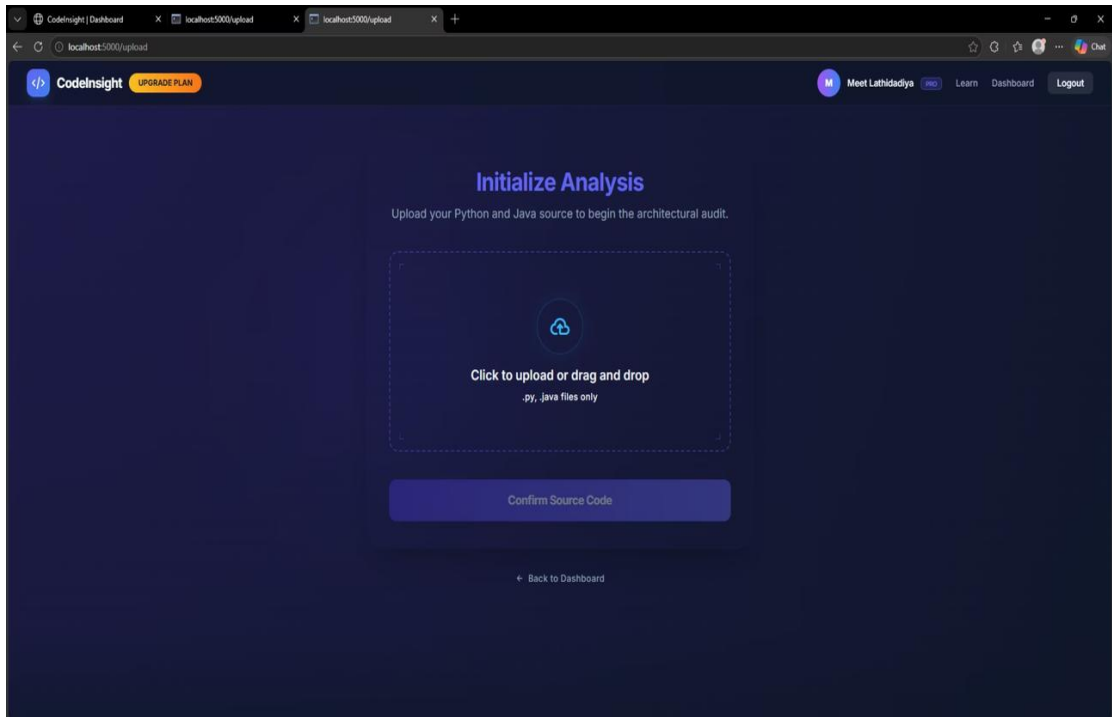
- **Forgot Password Page**



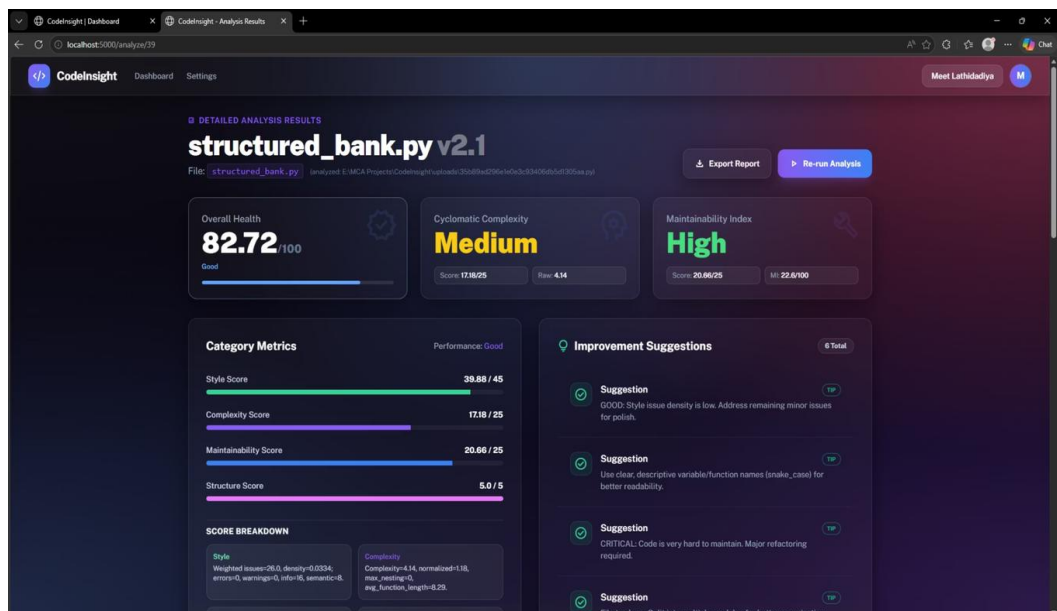
- **Dashboard**



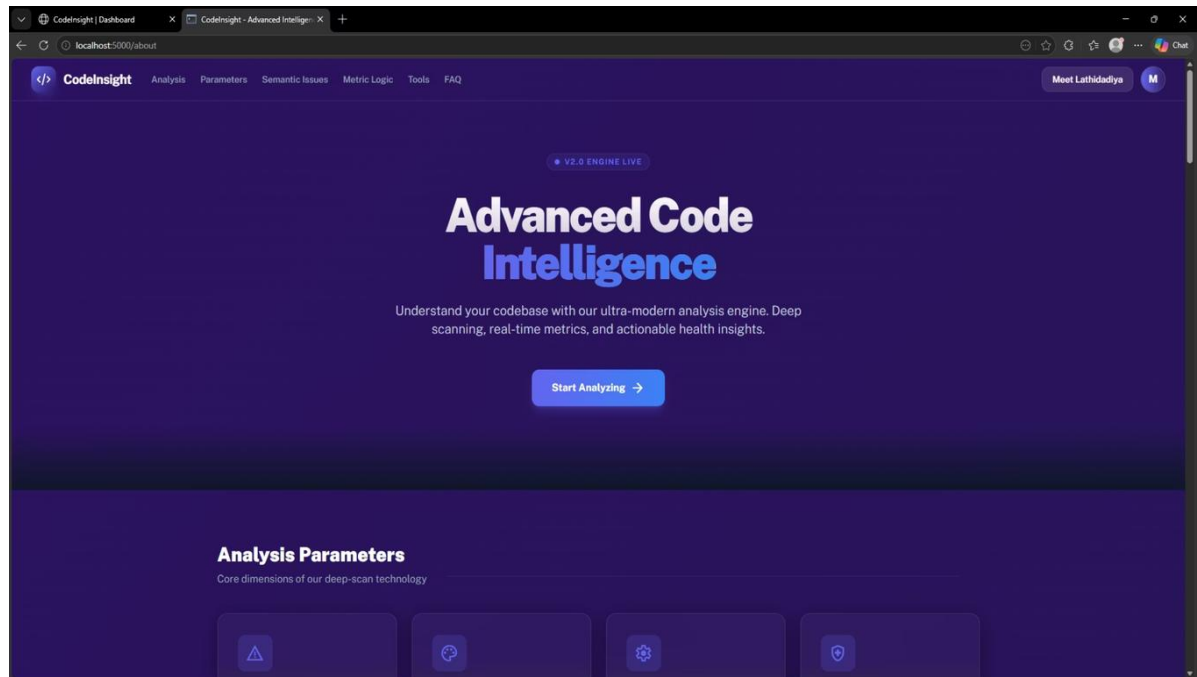
- Upload



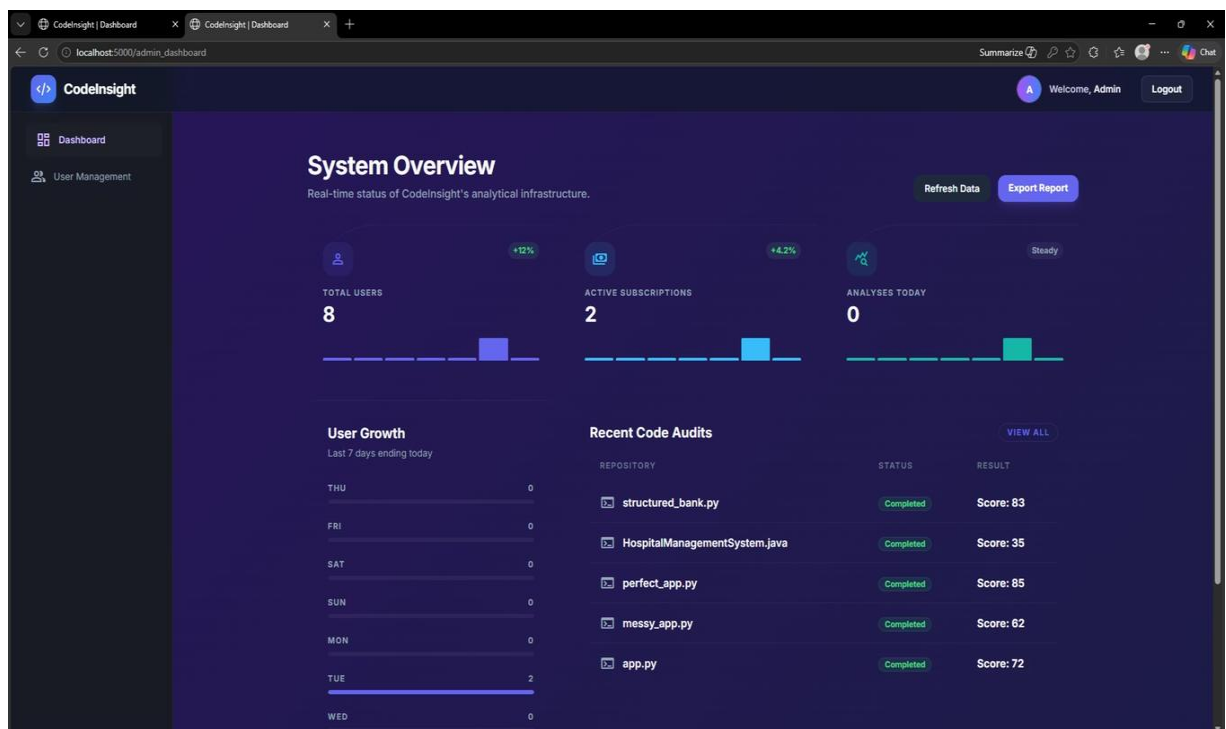
- Result



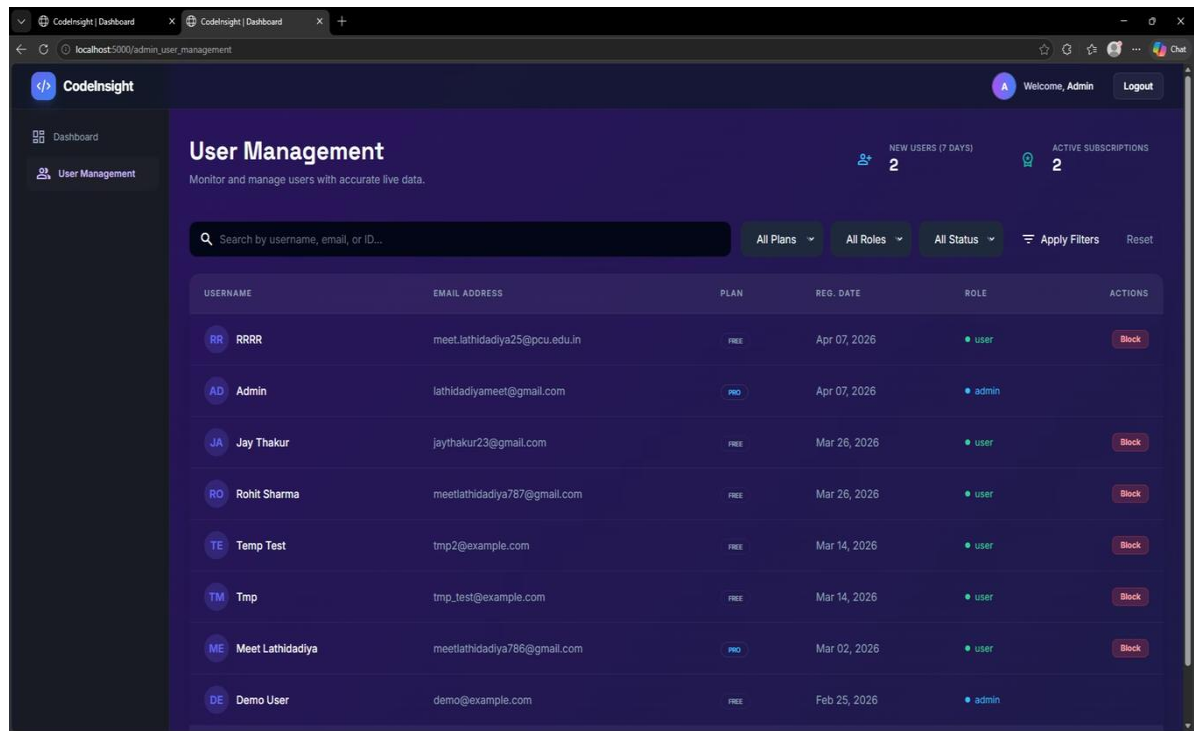
- About



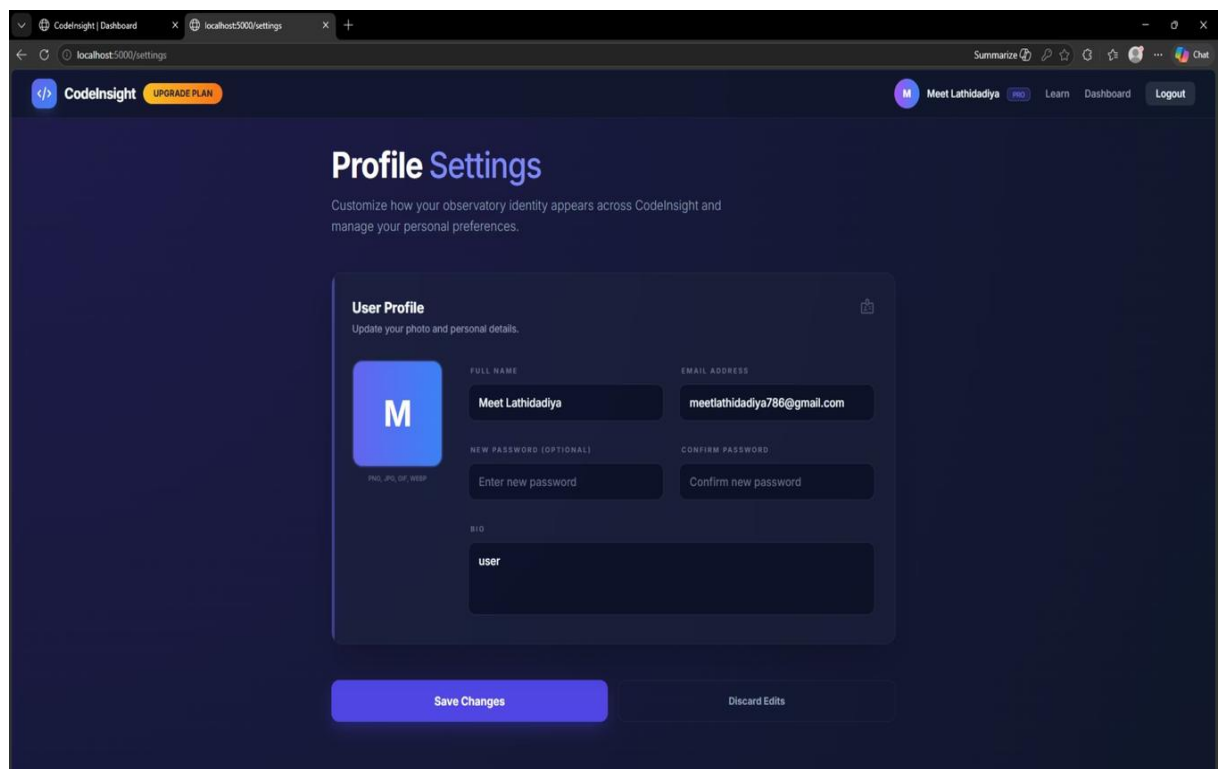
- Admin_Dashboard



- Admin_User_Management



- Settings



6. Testing

➤ Test Cases and Testing Methodology:

Test 1: Admin Login Page

- Project Name: CodeInsight
- Module Name: Admin

Test Summery:

Total Test Cases-04

Test Cases Passed-04

Test Cases Failed-00

Test Cases Executed-04

Test Result Table:

Test Case ID	Test Case Procedure	Input Data	Expected Output	Actual Output	Test Status
CI-ALG-01	Checking the functionality of Login	1. Enter valid username in textbox 2. Enter valid password in textbox	Admin dashboard should be displayed	Admin dashboard displayed	Pass
CI-ALG-02	Checking invalid login credentials	1. Enter invalid username 2. Enter invalid password	Error message should be displayed	Error message displayed	Pass
CI-ALG-03	Checking empty fields validation	1. Leave username and password empty.	Validation message should be displayed	Validation message displayed	Pass
CI-ALG-04	Checking blocked admin login	1. Enter blocked account details.	Should be denied	Login denied	Pass

Test 2: User Registration

- Project Name: CodeInsight
- Module Name: Authentication

Test Summery:

Total Test Cases-04

Test Cases Passed-04

Test Cases Failed-00

Test Cases Executed-04

Test Result Table:

Test Case ID	Test Case Procedure	Input Data	Expected Output	Actual Output	Test Status
CI- REG-01	Checking registration with valid data	Enter valid name, email, password and confirm password	Account should be created successfully	Account created successfully	Pass
CI- REG-02	Checking password mismatch	Enter password and confirm password as different	Error message should be displayed	Error message displayed	Pass
CI- REG-03	Checking duplicate email registration	Enter password and confirm password as different	Registration should be rejected	Registration rejected	Pass
CI- REG-04	Checking empty field validation	Leave required fields blank	Validation message should be displayed	Validation message displayed	Pass

Test 3: File Upload and Analysis Page

- Project Name: CodeInsight
- Module Name: File Upload and Analysis

Test Summery:

Total Test Cases-04

Test Cases Passed-04

Test Cases Failed-00

Test Cases Executed-04

Test Result Table:

Test Case ID	Test Case Procedure	Input Data	Expected Output	Actual Output	Test Status
CI- UPA-01	Upload valid Python or Java file	Select valid .py or .java file and click upload	File should be uploaded successfully	File uploaded successfully	Pass
CI- UPA-02	Upload invalid file type	Select .txt file and click upload	Error message should be displayed	Error message displayed	Pass
CI- UPA-03	Analyze uploaded file	Click analyze on uploaded file	Score and issues should be displayed	Score and issues displayed	Pass
CI- UPA-04	Upload empty file	Upload blank source file	Empty file warning should be shown	Warning shown	Pass

Test 4: Password Reset Page

- Project Name: CodeInsight
- Module Name: Authentication

Test Summery:

Total Test Cases-04

Test Cases Passed-04

Test Cases Failed-00

Test Cases Executed-04

Test Result Table:

Test Case ID	Test Case Procedure	Input Data	Expected Output	Actual Output	Test Status
CI- PR -01	Request password reset with valid email	Enter registered email and submit	Reset link should be generated/sent	Reset link generated/sent	Pass
CI- PR -02	Request password reset with invalid email	Enter unregistered email and submit	Generic success message should appear	Message displayed	Pass
CI- PR -03	Reset password with valid token	Open valid reset link and enter new password	Password should be updated successfully	Password updated successfully	Pass
CI- UPA-04	Reset password with invalid token	Open expired/invalid reset link	Error message should be displayed	Error message displayed	Pass

➤ **Bug Fixes/Issues Resolved**

1. Incorrect Scoring Calculation

Issue: The scoring algorithm produced very low scores for good-quality code due to improper formulas and excessive penalties.

Fix: Updated the scoring logic using normalized and continuous formulas, ensuring fair and accurate evaluation.

2. Complexity Score Too Low

Issue: The complexity score was overly penalized, even for moderate complexity values.

Fix: Adjusted the formula by normalizing complexity values to provide realistic scoring.

3. Maintainability

Issue: Maintainability scores were significantly lower than expected due to incorrect calculations and double penalties.

Fix: Corrected the formula and ensured penalties are applied only when necessary.

4. Score Miscalculation

Issue: Maintainability scores were significantly lower than expected due to incorrect calculations and double penalties.

Fix: Corrected the formula and ensured penalties are applied only when necessary.

5. Incorrect Suggestion Severity

Issue: The system displayed “SEVERE” warnings even when there were no major issues.

Fix: Implemented issue density-based severity classification (Good, Moderate, Severe).

7. Results and Discussion

➤ Outcomes of the project:

The development of the CodeInsight – Intelligent Code Analysis & Scoring System resulted in several important outcomes that demonstrate the effectiveness and usefulness of the system.

1. Automated Code Analysis System

A fully functional system was developed that can automatically analyze source code files and evaluate their quality based on multiple parameters such as syntax, style, complexity, and maintainability.

2. Unified Scoring Mechanism

A unique scoring system was implemented that combines different metrics into a single score out of 100, making it easier for users to understand overall code quality.

3. Multi-Level Analysis

The system successfully integrates multiple analysis techniques, including:

- Syntax checking using AST
- Style checking using Pylint
- Complexity and maintainability using Radon
- Custom semantic checks

4. Intelligent Suggestions

The system provides meaningful and actionable suggestions to help users improve their code quality and follow best practices.

5. User-Friendly Interface

A modern and interactive dashboard was developed to display results clearly, including score breakdowns, metrics, and suggestions.

6. Modular and Scalable Design

The system is built using a modular architecture, allowing easy maintenance and future enhancements such as adding support for more programming languages.

7. Report Generation

The system can generate detailed analysis reports, which can be viewed on-screen and optionally exported for documentation purposes.

8. Admin Control

An admin module enables monitoring of users, analysis activities, and system performance, improving system management.

9. Improved Learning and Productivity

The system helps students and developers understand code quality concepts and improves productivity by reducing manual code review effort.

➤ **Comparison with existing solutions:**

Feature / Criteria	Manual Code Review	CodeInsight (Proposed System)
Speed	Slow and time-consuming	Fast and automated
Accuracy	Depends on reviewer expertise	Consistent and rule-based
Scalability	Difficult for large codebases	Easily scalable
Cost	Requires human effort/resources	Low cost (automated system)
Consistency	May vary between reviewers	Highly consistent results
Error Detection	Can detect deep logical issues	Detects syntax, style, and patterns
Semantic Analysis	Strong (human understanding)	Moderate (rule-based + custom)
Unified Scoring	Not available	Available (score out of 100)
Feedback	Detailed but subjective	Instant and structured
Availability	Limited to working hours	Available anytime (24/7)
Learning Support	Depends on reviewer guidance	Provides automatic suggestions
Ease of Use	Requires experienced reviewers	Simple and user-friendly

➤ **Challenges Faced:**

During the development of the CodeInsight – Intelligent Code Analysis & Scoring System, several technical and design challenges were encountered. These challenges helped in improving problem-solving skills and refining the system.

1. Designing Accurate Scoring Algorithm

Challenge: Creating a fair scoring system that correctly evaluates code quality without being too harsh or too lenient.

Solution: Implemented normalized and continuous scoring formulas to balance different metrics.

2. Integration of Multiple Tools

Challenge: Combining tools like Pylint, Radon, and AST into a single workflow.

Solution: Developed a centralized analyzer module with proper error handling and fallback mechanisms.

3. Handling Different Programming Languages

Challenge: Supporting both Python and Java with different analysis techniques.

Solution: Created separate analysis engines for each language with a unified output format.

4. File Validation and Error Handling

Challenge: Managing invalid, empty, or unsupported file uploads.

Solution: Added validation checks and user-friendly error messages.

5. Ensuring Accurate Maintainability and Complexity Metrics

Challenge: Incorrect or inconsistent metric calculations during early stages.

Solution: Improved logic and used reliable libraries like Radon.

8. Conclusion and Future Scope

➤ **Summary of the Work**

The **CodeInsight – Intelligent Code Analysis & Scoring System** was successfully developed to automate the process of evaluating code quality. The system analyzes source code using multiple techniques such as syntax checking, style analysis, complexity measurement, and maintainability evaluation. It integrates tools like AST, Pylint, and Radon along with custom semantic checks to provide a comprehensive analysis.

A unified scoring system was implemented to combine different metrics into a single score, making it easier for users to understand code quality. The system also provides meaningful suggestions and a user-friendly dashboard, helping developers and students improve their coding practices efficiently.

➤ **Limitations**

Despite its effectiveness, the system has certain limitations:

- Currently supports only limited programming languages (mainly Python and Java)
- Java analysis is based on heuristic methods and is less accurate than Python analysis
- Semantic analysis is limited and may not detect all logical issues
- Does not fully replace manual code review for complex scenarios
- Performance may decrease for very large files
- Limited integration with external tools or IDEs

➤ **Future Enhancements**

The system can be further improved with the following enhancements:

- Support for additional programming languages such as JavaScript, C++, and others
- Integration of AI/ML techniques for advanced code analysis and suggestions
- Development of a plugin for IDEs like VS Code for real-time analysis
- Improved semantic analysis for deeper logic understanding
- Cloud-based deployment for scalability and accessibility
- Enhanced visualization with charts and analytics dashboards
- Integration with version control systems (e.g., GitHub)
- Advanced reporting features and customization options

The project successfully developed an automated code analysis system with unified scoring and suggestions. However, it has limitations such as limited language support and basic semantic analysis. Future enhancements include AI-based analysis, support for more languages, and integration with IDEs and cloud platforms.

References

Source/Tool	Documentation Link
Flask	https://flask.palletsprojects.com/
Werkzeug	https://werkzeug.palletsprojects.com/
Flask-Mail	https://pythonhosted.org/Flask-Mail/
Pylint	https://pylint.readthedocs.io/
Radon	https://radon.readthedocs.io/
ReportLab	https://www.reportlab.com/documentation/
MySQL 8.0	https://dev.mysql.com/doc/refman/8.0/en/
Python 3 (ast module)	https://docs.python.org/3/library/ast.html
Javalang	https://github.com/c2nes/javalang
Bootstrap	https://getbootstrap.com/docs/
Chart.js	https://www.chartjs.org/docs/latest/
OWASP Top 10	https://owasp.org/www-project-top-ten/